

OOSE - Mod 5

Software Testing and Maintenance

Testing is a set of activities that can be planned in advance and conducted systematically. For this reason a template for software testing—a set of steps into which you can place specific test case design techniques and testing methods—should be defined for the software process.

A number of software testing strategies have been proposed in the literature. All provide you with a template for testing and all have the following generic characteristics:

- To perform effective testing, you should conduct effective technical reviews. By doing this, many errors will be eliminated before testing commences.
- Testing begins at the component level and works “outward” toward the integration of the entire computer-based system.
- Different testing techniques are appropriate for different software engineering approaches and at different points in time.
- Testing is conducted by the developer of the software and (for large projects) an independent test group.
- Testing and debugging are different activities, but debugging must be accommodated in any testing strategy.

A strategy for software testing must accommodate low-level tests that are necessary to verify that a small source code segment has been correctly implemented as well as high-level tests that validate major system functions against customer requirements. A strategy should provide guidance for the practitioner and a set of milestones for the manager. Because the steps of the test strategy occur at a time when deadline pressure begins to rise, progress must be measurable and problems should surface as early as possible.

The goal of testing is to find errors, and a good test is one that has a high probability of finding an error. Therefore, you should design and implement a computer based system or a product with “testability” in mind. At the same time, the tests themselves must exhibit a set of characteristics that achieve the goal of finding the most errors with a minimum of effort.

Black-box testing alludes to tests that are conducted at the software interface. A black-box test examines some fundamental aspect of a system with little regard for the internal logical structure of the software.

White-box testing of software is predicated on close examination of procedural detail. Logical paths through the software and collaborations between components are tested by exercising specific sets of conditions and/or loops.

The construction of object-oriented software begins with the creation of requirements (analysis) and design models.¹ Because of the evolutionary nature of the OO software engineering paradigm, these models begin as relatively informal representations of system requirements and evolve into detailed models of classes, class relationships, system design and allocation, and object design (incorporating a model of object connectivity via messaging). At each stage, the models can be “tested” in an attempt to uncover errors prior to their propagation to the next

iteration. It can be argued that the review of OO analysis and design models is especially useful because the same semantic constructs (e.g., classes, attributes, operations, messages) appear at the analysis, design, and code levels. Therefore, a problem in the definition of class attributes that is uncovered during analysis will circumvent side effects that might occur if the problem were not discovered until design or code (or even the next iteration of analysis).

For example, consider a class in which a number of attributes are defined during the first iteration of analysis. An extraneous attribute is appended to the class (due to a misunderstanding of the problem domain). Two operations are then specified to manipulate the attribute. A review is conducted and a domain expert points out the problem. By eliminating the extraneous attribute at this stage, the following problems and unnecessary effort may be avoided during analysis:

1. Special subclasses may have been generated to accommodate the unnecessary attribute or exceptions to it. Work involved in the creation of unnecessary subclasses has been avoided.
2. A misinterpretation of the class definition may lead to incorrect or extraneous class relationships.
3. The behavior of the system or its classes may be improperly characterized to accommodate the extraneous attribute.

OO TESTING

Object-Oriented Testing focuses on verifying the correctness and performance of software systems developed using object-oriented programming (OOP) principles such as encapsulation, inheritance, polymorphism, and abstraction.

Unit Testing

Unit testing is a fundamental software testing technique where individual units or components of a software application are tested in isolation to ensure that each part functions as expected. In object-oriented programming, a "unit" typically refers to a class or a method. The goal of unit testing is to verify that each part of the program performs correctly under a variety of conditions, independently of other parts. This level of testing is considered the foundation of the testing pyramid, providing fast feedback and early defect detection.

In object-oriented systems, unit testing involves creating test cases for classes, specifically focusing on their attributes (data members), constructors, and public methods. Since object behavior is encapsulated, unit testing usually interacts with an object only through its public interface. This ensures that the internal implementation details are not exposed or directly tested, adhering to the principle of encapsulation.

Unit testing also emphasizes the importance of testing object states and behaviors. For example, the tests should check whether a method correctly modifies the object's internal state or returns the expected output for a given input. Moreover, unit tests must validate both normal behavior (e.g. correct input) and edge cases (e.g. invalid input or unexpected usage), ensuring robustness and reliability of the code.

To isolate the unit under test, mock objects, stubs, or fakes are often used for dependencies such as databases, APIs, or other classes. This isolation ensures that the test results are not influenced

by external factors and focus purely on the logic within the unit being tested. For example, if a class depends on a database connection, a mock database object might be injected to simulate its behavior without involving real data.

Unit testing also plays a crucial role in regression testing, where previously written tests ensure that new changes don't break existing functionality. This encourages developers to refactor and evolve code with confidence. Additionally, comprehensive unit testing improves code maintainability,

facilitates documentation (through test case scenarios), and leads to better-designed code due to the practice of Test-Driven Development (TDD)—where tests are written before the actual code. Despite its strengths, unit testing also has limitations. It cannot detect integration issues, system-level defects, or performance problems. Furthermore, writing effective unit tests requires time and discipline, especially when dealing with tightly coupled or poorly designed code, which may necessitate significant refactoring.

Integration Testing

Integration testing is a critical phase in the software testing life cycle where individual software modules—previously verified through unit testing—are combined and tested as a group to evaluate their interaction and data exchange. While unit testing ensures each component works in isolation, integration testing focuses on verifying that modules work together correctly and cohesively as part of a larger system.

In object-oriented software, integration testing examines the interactions between classes, interfaces, and modules. For example, if Class A calls a method in Class B, integration testing verifies that the method call happens correctly, with expected inputs and outputs, and that any dependencies between the two behave as intended. This type of testing ensures the software components integrate smoothly without issues like incorrect data passing, broken logic flows, or unhandled exceptions. There are several common approaches to integration testing:

1. Top-Down Integration Testing:

In this approach, testing begins from the top-level modules and gradually progresses to lower-level modules. Stubs (simplified versions of lower-level modules) are often used to simulate the behavior of missing components. This strategy is useful when the higher-level logic is ready but the lower-level modules are still under development.

2. Bottom-Up Integration Testing:

Here, testing starts with the lower-level modules, which are combined and tested first. Drivers (dummy modules used to simulate higher-level modules) are created to call and test these components. This is helpful when the lower-level modules are stable and can be tested independently before integrating them into higher-level components.

3. Big Bang Integration Testing:

In this approach, all modules are integrated simultaneously and the entire system is tested as a whole. While this can be quicker in smaller systems, it makes it harder to isolate defects and understand their root causes, especially when many components are involved.

4. Incremental Integration Testing:

This combines the benefits of top-down and bottom-up approaches by integrating and

testing one module at a time. As each new module is added, it's tested in combination with already integrated modules. This method makes it easier to identify integration issues early.

During integration testing, interface contracts between components are of primary importance. These contracts define how different modules exchange data—through function calls, shared data structures, or APIs. Errors in integration often stem from mismatched data formats, incorrect API usage, missing error handling, or assumptions about component behavior.

In modern development environments, especially with service-oriented architectures and microservices, integration testing plays a vital role in ensuring services can communicate over networks through APIs. API testing (often a part of integration testing) validates request-response mechanisms, authentication, and data handling across services.

Tools like Postman, JUnit (with integration test annotations), TestNG, and automated frameworks like Selenium (for web app flows) help implement integration testing. In Continuous Integration/Continuous Deployment (CI/CD) pipelines, integration tests are often run after unit tests to quickly identify any newly introduced integration failures.

System Testing

System Testing is a high-level testing phase where the entire software system is tested as a whole, in an environment that closely mirrors the actual production setting. Unlike unit or integration testing, which focuses on individual components or their interactions, system testing validates the complete and integrated software application to ensure it meets the specified requirements.

System testing is a black-box testing technique, meaning that the testers do not need knowledge of the internal code or architecture. Instead, they focus on inputs and expected outputs and verify that the system behaves correctly in real-world scenarios. It tests both functional and non-functional aspects of the software, such as performance, security, usability, and compatibility.

Functional Testing

System testing verifies the system against the functional requirements (i.e., what the system should do). This includes validating workflows, inputs and outputs, user interactions, and business logic. Test cases are derived from requirement documents, use cases, and user stories. Each function of the application is tested by providing appropriate input and verifying the output.

Non-Functional Testing:

System testing also involves testing non-functional requirements, such as:

- Performance Testing: Checks how the system behaves under load or stress.
- Security Testing: Ensures the system is secure from external threats or vulnerabilities.
- Usability Testing: Evaluates user interface and user experience.
- Compatibility Testing: Ensures the software works on different devices, browsers, OSs.
- Reliability and Availability Testing: Checks system stability and uptime.

End-to-End Workflow Verification:

In this phase, testers simulate real-life user scenarios, ensuring that the entire user journey works seamlessly—from logging in to completing transactions. This validates not only individual modules but also their integration and how they behave collectively within a production-like environment.

Types of Tests in System Testing

- Smoke Testing: A quick check to ensure the basic functionalities work.
- Sanity Testing: Focused testing on specific functionalities after changes.
- Regression Testing: Ensures new code changes haven't broken existing features.
- Interface Testing: Verifies that interfaces between components work correctly.
- Recovery Testing: Tests the system's ability to recover from crashes or hardware failures.
- Installation Testing: Verifies that the software installs and uninstalls correctly.

Importance of System Testing

System testing is critical because it is the first phase where the entire system is validated from an end-user's perspective. It uncovers issues that cannot be detected in earlier testing phases, such as configuration problems, environment-related errors, and full workflow mismatches.

It is usually performed by a dedicated QA/testing team, not the developers, to ensure objectivity. This phase acts as a gate before the software proceeds to acceptance testing, where actual users validate it.

In essence, system testing is a comprehensive validation of the whole software system, ensuring it meets the functional and non-functional requirements defined during the planning phase. It's the most complete form of testing before the software is released to users and plays a vital role in delivering high-quality, reliable, and user-friendly products.

Acceptance Testing

Acceptance Testing is the final level of testing performed before a software product is delivered to the customer or moved into production. It is primarily conducted to determine whether the system meets the business requirements and is ready for deployment. This phase ensures that the software is acceptable for release, functioning as expected for real-world use.

Unlike system testing—which is conducted by the internal testing team—acceptance testing is often performed by the client, end-users, or a dedicated user acceptance testing (UAT) team. The goal is to validate the system from a user's perspective, verifying that the software satisfies their needs and behaves correctly in practical scenarios.

Types of Acceptance Testing

1. User Acceptance Testing (UAT):
 - Conducted by actual users to ensure the system works in real-life situations.
 - Verifies workflows, data handling, and user interface from a usability and business point of view.
 - Users test with real or realistic data to simulate actual tasks.
2. Business Acceptance Testing (BAT):

- Focuses on whether the software meets the business goals and objectives.
 - Often performed by business analysts or stakeholders to ensure the solution aligns with strategic needs.
3. Contract Acceptance Testing:
 - Verifies that the software meets the criteria set forth in a contract.
 - Often used in vendor-client relationships to validate delivery obligations before final payment or sign-off.
 4. Regulation Acceptance Testing (Compliance Testing):
 - Ensures the software complies with legal, regulatory, or industry standards, such as GDPR, HIPAA, or ISO requirements.
 5. Alpha Testing:
 - Conducted by internal staff at the developer's site before releasing the product to external users.
 - May uncover issues not found in earlier testing stages.
 6. Beta Testing:
 - Conducted by a limited number of end users in a real environment.
 - Helps identify unexpected bugs, usability issues, or performance concerns before full-scale release.

Objectives of Acceptance Testing

- Confirm the software system fulfills all user requirements.
- Identify bugs or issues that were not caught in earlier stages.
- Ensure the application works correctly in realistic user environments.
- Gain stakeholder confidence in the product before going live.
- Provide the final sign-off for software deployment or release.

Acceptance Testing Process

1. Requirement Analysis: Identify which business requirements need to be validated.
2. Test Planning: Define the scope, entry/exit criteria, and schedule.
3. Test Case Design: Prepare user-focused test scenarios and scripts.
4. Test Execution: Users or clients execute the test cases using real-world workflows.
5. Defect Reporting and Retesting: Log any issues found, fix them, and retest.
6. Sign-off and Approval: Once users are satisfied, they approve the system for release.

Acceptance testing is the final checkpoint before a product goes live. It validates the software against business goals, user expectations, and contractual obligations. Successful acceptance testing means the software is ready for the production environment and real-world use. It's a crucial phase for building trust between the development team and the client, as it confirms that the final product is both functional and valuable.

Test Case Design

Test case design is a fundamental part of object-oriented software testing, aiming to verify the correctness, reliability, and performance of individual classes and their interactions. In OO systems, test cases must be carefully crafted to reflect the dynamic behavior of objects, methods, and their interactions over time.

Each test case should be uniquely identified with a test ID or name and should be explicitly linked to a specific class or method being tested. This traceability ensures that test coverage is comprehensive and aligned with the system's structure and behavior.

Key Elements of a Good OO Test Case:

1. **Test Case ID & Class Association:** Each test case must have a unique identifier and must be clearly associated with the class or classes it aims to test. This helps in organizing and managing the test suite efficiently.
2. **Purpose of the Test:** The objective or intent of the test must be clearly stated. This describes what aspect of the class behavior is being validated—such as method functionality, constructor behavior, object interaction, or exception handling.
3. **Input Data and Preconditions:** All inputs, initial values, and object states must be defined before execution. If the method depends on object states or prior actions, those should be specified as part of the test setup.
4. **Sequence of Operations:** OO testing focuses not just on individual methods but on the sequence of method calls and interactions. Therefore, test cases must include a step-by-step procedure—including object creation, method invocation, and the order of execution—to simulate real-world usage.
5. **Expected Results:** For each operation, the expected outcome (return value, state change, exception, etc.) must be specified. This enables the tester to compare actual outcomes with expected ones and determine pass/fail status.
6. **Postconditions and Clean-Up:** Describe the expected object state after test execution and any necessary cleanup activities, especially in environments with shared resources or persistent data.

Example Test Case Structure for an OO System:

Field	Description
Test Case ID	TC001
Associated Class	BankAccount
Purpose	To verify that deposit() correctly updates the account balance.
Preconditions	A BankAccount object is instantiated with an initial balance of \$1000.
Test Steps	1. Call deposit(500) 2. Retrieve balance using getBalance()
Expected Output	The returned balance should be \$1500.
Postconditions	Account balance remains \$1500 unless further transactions are performed.

In object-oriented testing, test case design goes beyond function-level validation and includes testing interactions, message passing, object states, and method sequences. Well-structured test cases ensure clarity, traceability, and thorough coverage. A strong focus on designing appropriate sequences of operations that reflect real-life object interactions is key to effective OO test design.

Testing Methods at Class Level

Random Testing : requires large number of data permutation and combination

- Identify operations applicable to a class
- Define constraints on their use– e.g. the class must always be initialized first
- Identify a minimum test sequence – an operation sequence that defines the minimum life history of the class
- Generate a variety of random test sequences– this exercises more complex class instance life histories

Example of Random Testing

1. consider an account class that has the following operations:
open, setup, deposit, withdrawal, balance, summarize, credit Limit, close
2. The account must be opened first and closed on completion
Minimum behavioral life history of account may occur: open • setup • deposit • withdrawal • close
3. a variety of behaviors may occur: open • setup • deposit • [deposit | withdrawal | balance | summarize | credit limit]ⁿ • withdrawal • close
4. different operation sequences can be generated randomly:
 - test case #1: open•setup•deposit•deposit• balance• summarize• withdrawal•close
 - test case #2: open•setup•deposit•withdrawal• deposit• balance• credit limit•withdrawal•close

These and other random order tests are conducted to exercise different class instance life histories.

Partition Testing: reduces the number of test cases required to test a class

- State-based partitioning
 - Categorizes class operations based on their ability to change the state of the class.
 - Tests designed in way so that operations that cause state changes and those that do not change state are tested separately

Example:

- Test case #sp1: open• setup• deposit• deposit• withdrawal• withdrawal• close
- Test case #sp2: open• setup•summarize • credit limit• close

The tests are derived from the above state and transition and below are the possible scenarios that need to be tested.

Tests	Test 1	Test 2	Test 3
Start State	Off	On	On
Input	Switch ON	Switch Off	Switch off
Output	Light ON	Light Off	Fault
Finish State	ON	OFF	On

Attribute-based partitioning

- Categorizes class operations based on the attributes that they use and modify
- Tests designed in way so that operations that use or modify attributes are tested separately from those that do not use or modify attributes

Example:

For the account class, the attributes balance and credit limit can be used to define operations. Operations are divided into three partitions:

1. operations that use credit limit
2. operations that modify credit limit
3. operations that do not use or modify credit limit

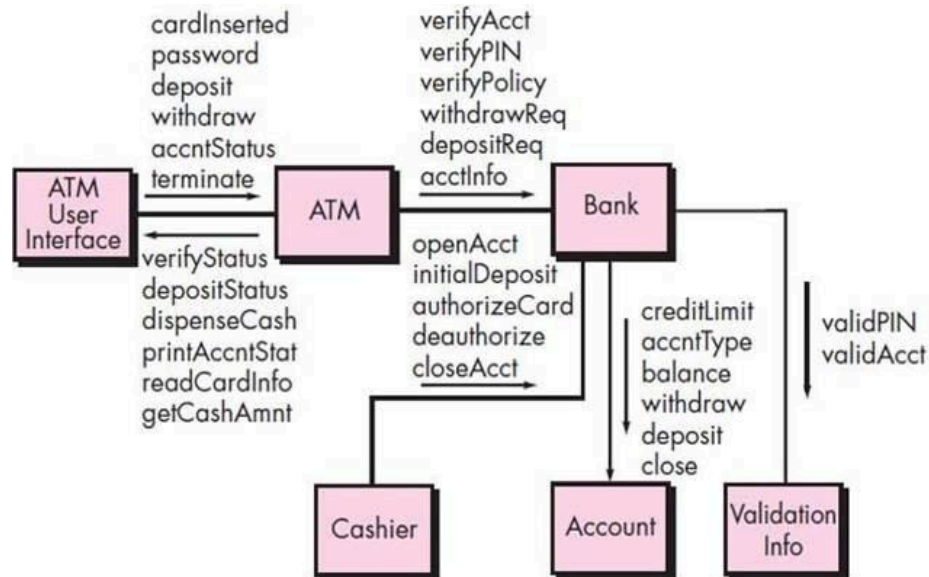
Category-based partitioning

- Categorizes class operations based on the generic function that each performs such as initialization, computation, query, termination

Example: operations in the account class can be categorized as:

- initialization (open, setup)
- computation (deposit, withdraw)
- queries (balance, summarize, credit limit)
- termination (close)

Interclass Test Case Design: Test case design becomes more complicated as integration of the OO system begins – testing of collaboration between classes



Multiple class testing

- For each client class, use the list of class operators to generate a series of random test sequences that send messages to other server classes
- For each message generated, determine the collaborator class and the corresponding operator in the server object
- For each operator in the server object (invoked by a client object message), determine the message that it transmits
- For each of the message, determine the next level of operators that are invoked and

incorporate these into the test sequence.

Test derived from behavior models

- Use the state transition diagram as a model
- Test cases must cover all states
- Dynamic behavior of class

Validation Testing in an OO Context

At the validation or system level, the details of class connections disappear. Like conventional validation, the validation of OO software focuses on user-visible actions and user-recognizable outputs from the system. To assist in the derivation of validation tests, the tester should draw upon use cases that are part of the requirements model. The use case provides a scenario that has a high likelihood of uncovered errors in user-interaction requirements.

Conventional black-box testing methods can be used to drive validation tests. In addition, you may choose to derive test cases from the object behavior model and from an event flow diagram created as part of OOA.

Correctness of OOA and OOD Models

The notation and syntax used to represent analysis and design models will be tied to the specific analysis and design methods that are chosen for the project. Hence syntactic correctness is judged on proper use of the symbology; each model is reviewed to ensure that proper modeling conventions have been maintained. During analysis and design, you can assess semantic correctness based on the model's conformance to the real-world problem domain. If the model accurately reflects the real world (to a level of detail that is appropriate to the stage of development at which the model is reviewed), then it is semantically correct. To determine whether the model does, in fact, reflect real-world requirements, it should be presented to problem domain experts who will examine the class definitions and hierarchy for omissions and ambiguity. Class relationships (instance connections) are evaluated to determine whether they accurately reflect real-world object connections.

class name: credit sale	
class type: transaction event	
class characteristics: nontangible, atomic, sequential, permanent, guarded	
responsibilities:	collaborators:
read credit card	credit card
get authorization	credit authority
post purchase amount	product ticket
	sales ledger
	audit file
generate bill	bill

CRC Card

Object Oriented Testing Methods:

The architecture of object-oriented software results in a series of layered subsystems that encapsulate collaborating classes. Each of these system elements (subsystems and classes) performs functions that help to achieve system requirements. It is necessary to test an OO system at a variety of different levels in an effort to uncover errors that may occur as classes collaborate with one another and subsystems communicate across architectural layers.

Test-case design methods for object-oriented software continue to evolve. However, an overall approach to OO test-case design has been suggested:

1. Each test case should be uniquely identified and explicitly associated with the class to be tested.
2. The purpose of the test should be stated.
3. A list of testing steps should be developed for each test and should contain:
 - a. A list of specified states for the class that is to be tested
 - b. A list of messages and operations that will be exercised as a consequence of the test
 - c. A list of exceptions that may occur as the class is tested
 - d. A list of external conditions (i.e., changes in the environment external to the software that must exist in order to properly conduct the test)
 - e. Supplementary information that will aid in understanding or implementing the test

Unlike conventional test-case design, which is driven by an input-process-output view of software or the algorithmic detail of individual modules, object-oriented testing focuses on designing appropriate sequences of operations to exercise the states of a class.

Software Maintenance:

The underlying laws that have been derived are worthy of note:

The Law of Continuing Change: Software that has been implemented in a real world computing context and will therefore evolve over time (called E-type systems) must be continually adapted else they become progressively less satisfactory.

The Law of Increasing Complexity: As an E-type system evolves its complexity increases unless work is done to maintain or reduce it.

The Law of Self Regulation: The E-type system evolution process is self regulating with distribution of product and process measures close to normal.

The Law of Conservation of Organizational Stability: The average effective global activity rate in an evolving E-type system is invariant over product lifetime.

The Law of Conservation of Familiarity: As an E-type system evolves all associated with it, developers, sales personnel, users, for example, must maintain mastery of its content and behavior to achieve satisfactory evolution. Excessive growth diminishes that mastery. Hence the average incremental growth remains invariant as the system evolves.

The Law of Continuing Growth: The functional content of E-type systems must be continually

increased to maintain user satisfaction over their lifetime.

The Law of Declining Quality: The quality of E-type systems will appear to be declining unless they are rigorously maintained and adapted to operational environment changes.

The Feedback System Law: E-type evolution processes constitute multilevel, multi-loop, multi-agent feedback systems and must be treated as such to achieve significant improvement over any reasonable base.

Software Maintenance:

It begins almost immediately. Software is released to end users, and within days, bug reports filter back to the software engineering organization. Within weeks, one class of users indicates that the software must be changed so that it can accommodate the special needs of their environment. And within months, another corporate group who wanted nothing to do with the software when it was released now recognizes that it may provide them with unexpected benefits. They'll need a few enhancements to make it work in their world.

The challenge of software maintenance has begun. You're faced with a growing queue of bug fixes, adaptation requests, and outright enhancements that must be planned, scheduled, and ultimately accomplished. Before long, the queue has grown long and the work it implies threatens to overwhelm the available resources. As time passes, your organization finds that it's spending more money and time maintaining existing programs than it is engineering new applications. In fact, it's not unusual for a software organization to expend as much as 60 to 70 percent of all resources on software maintenance.

Maintainable software exhibits effective modularity. It makes use of design patterns that allow ease of understanding. It has been constructed using well-defined coding standards and conventions, leading to source code that is self-documenting and understandable. It has undergone a variety of quality assurance techniques that have uncovered potential maintenance problems before the software is released. It has been created by software engineers who recognize that they may not be around when changes must be made. Therefore, the design and implementation of the software must "assist" the person who is making the change.

Types of Software Maintenance

Corrective Maintenance

This involves fixing bugs, errors, and faults discovered after the software's release. It addresses problems that affect the software's functionality and is typically reactive, responding to user-reported issues or detected faults to restore proper operation quickly.

Adaptive Maintenance

This type modifies the software to keep it compatible with changing environments such as new operating systems, hardware upgrades, or changes in regulations and policies. It ensures the software continues to function correctly as its external environment evolves.

Perfective Maintenance

Perfective maintenance focuses on improving the software by enhancing performance, adding new features, refining usability, and restructuring code to increase maintainability. It is driven by

user feedback and changing business needs to keep the software relevant and efficient.

Preventive Maintenance

Preventive maintenance aims to anticipate and prevent future problems by optimizing the software, updating documentation, and implementing measures like backups and system reviews. It is proactive, designed to reduce the risk of latent faults becoming actual issues.

These four categories are widely recognized and standardized, for example, by ISO/IEC 14764, and together they cover the spectrum of activities needed to keep software operational and aligned with user and business requirements throughout its lifecycle.

Software Reengineering:

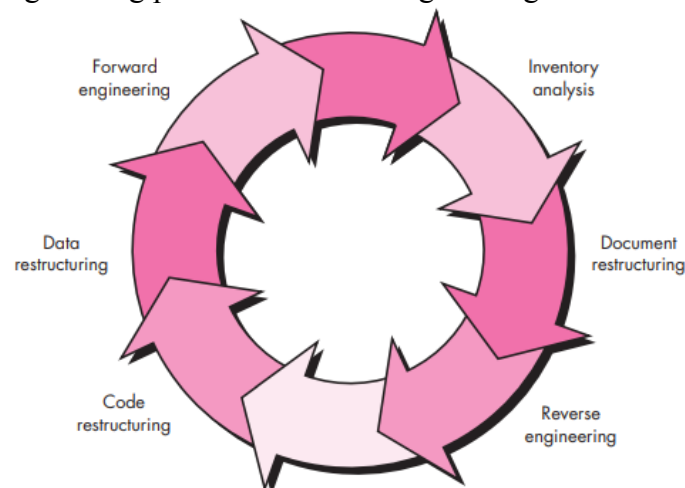
Reengineering strives to break away from the old rules about how we organize and conduct our business.

An application has served the business needs of a company for 10 or 15 years. During that time it has been corrected, adapted, and enhanced many times. People approached this work with the best intentions, but good software engineering practices were always shunted to the side (due to the press of other matters). Now the application is unstable. It still works, but every time a change is attempted, unexpected and serious side effects occur. Yet the application must continue to evolve. What to do?

Unmaintainable software is not a new problem. In fact, the broadening emphasis on software reengineering has been spawned by software maintenance problems that have been building for more than four decades.

A Software Reengineering Process Model

Reengineering takes time, it costs significant amounts of money, and it absorbs resources that might be otherwise occupied on immediate concerns. For all of these reasons, reengineering is not accomplished in a few months or even a few years. Reengineering of information systems is an activity that will absorb information technology resources for many years. That's why every organization needs a pragmatic strategy for software reengineering. A workable strategy is encompassed in a reengineering process model. Reengineering is a rebuilding activity.



Inventory analysis. Every software organization should have an inventory of all applications. The inventory can be nothing more than a spreadsheet model containing information that provides a detailed description (e.g., size, age, business criticality) of every active application. By sorting this information according to business criticality, longevity, current maintainability and supportability, and other locally important criteria, candidates for reengineering appear. Resources can then be allocated to candidate applications for reengineering work. It is important to note that the inventory should be revisited on a regular cycle. The status of applications (e.g., business criticality) can change as a function of time, and as a result, priorities for reengineering will shift.

Document restructuring. Weak documentation is the trademark of many legacy systems.

1. Creating documentation is far too time consuming. If the system works, you may choose to live with what you have. In some cases, this is the correct approach. It is not possible to re-create documentation for hundreds of computer programs. If a program is relatively static, is coming to the end of its useful life, and is unlikely to undergo significant change, let it be!
2. Documentation must be updated, but your organization has limited resources. You'll use a "document when touched" approach. It may not be necessary to fully document an application. Rather, those portions of the system that are currently undergoing change are fully documented. Over time, a collection of useful and relevant documentation will evolve.
3. The system is business critical and must be fully documented. Even in this case, an intelligent approach is to pare documentation to an essential minimum.

Each of these options is viable. Your software organization must choose the one that is most appropriate for each case.

Reverse engineering. The term reverse engineering has its origins in the hardware world. A company disassembles a competitive hardware product in an effort to understand its competitor's design and manufacturing "secrets." These secrets could be easily understood if the competitor's design and manufacturing specifications were obtained. But these documents are proprietary and unavailable to the company doing the reverse engineering. In essence, successful reverse engineering derives one or more design and manufacturing specifications for a product by examining actual specimens of the product.

Reverse engineering for software is quite similar. In most cases, however, the program to be reverse engineered is not a competitor's. Rather, it is the company's own work (often done many years earlier). The "secrets" to be understood are obscure because no specification was ever developed. Therefore, reverse engineering for software is the process of analyzing a program in an effort to create a representation of the program at a higher level of abstraction than source code. Reverse engineering is a process of design recovery. Reverse engineering tools extract data, architectural, and procedural design information from an existing program.

Code restructuring. The most common type of reengineering (actually, the use of the term reengineering is questionable in this case) is code restructuring. Some legacy systems have a relatively solid program architecture, but individual modules were coded in a way that makes

them difficult to understand, test, and maintain. In such cases, the code within the suspect modules can be restructured. To accomplish this activity, the source code is analyzed using a restructuring tool. Violations of structured programming constructs are noted and code is then restructured (this can be done automatically) or even rewritten in a more modern programming language. The resultant restructured code is reviewed and tested to ensure that no anomalies have been introduced. Internal code documentation is updated.

Data restructuring. A program with weak data architecture will be difficult to adapt and enhance. In fact, for many applications, information architecture has more to do with the long-term viability of a program than the source code itself. Unlike code restructuring, which occurs at a relatively low level of abstraction, data restructuring is a full-scale reengineering activity. In most cases, data restructuring begins with a reverse engineering activity. Current data architecture is dissected, and necessary data models are defined. Data objects and attributes are identified, and existing data structures are reviewed for quality. When data structure is weak (e.g., flat files are currently implemented, when a relational approach would greatly simplify processing), the data are reengineered. Because data architecture has a strong influence on program architecture and the algorithms that populate it, changes to the data will invariably result in either architectural or code-level changes.

Forward engineering. In an ideal world, applications would be rebuilt using an automated “reengineering engine.” The old program would be fed into the engine, analyzed, restructured, and then regenerated in a form that exhibited the best aspects of software quality. In the short term, it is unlikely that such an “engine” will appear, but vendors have introduced tools that provide a limited subset of these capabilities that addresses specific application domains (e.g., applications that are implemented using a specific database system). More importantly, these reengineering tools are becoming increasingly more sophisticated. Forward engineering not only recovers design information from existing software but uses this information to alter or reconstitute the existing system in an effort to improve its overall quality. In most cases, reengineered software reimplements the function of the existing system and also adds new functions and/or improves overall performance.

Reverse Engineering:

Reverse engineering conjures an image of the “magic slot.” You feed a haphazardly designed, undocumented source file into the slot and out the other end comes a complete design description (and full documentation) for the computer program. Unfortunately, the magic slot doesn’t exist. Reverse engineering can extract design information from source code, but the abstraction level, the completeness of the documentation, the degree to which tools and a human analyst work together, and the directionality of the process are highly variable.

The abstraction level of a reverse engineering process and the tools used to effect it refers to the sophistication of the design information that can be extracted from source code. Ideally, the abstraction level should be as high as possible. That is, the reverse engineering process should be capable of deriving procedural design representations (a low-level abstraction), program and data structure information (a somewhat higher level of abstraction), object models, data and/or control flow models (a relatively high level of abstraction), and entity relationship models (a high level of abstraction). As the abstraction level increases, you are provided with information that will allow easier understanding of the program. The completeness of a reverse engineering process

refers to the level of detail that is provided at an abstraction level. In most cases, the completeness decreases as the abstraction level increases. For example, given a source code listing, it is relatively easy to develop a complete procedural design representation. Simple architectural design representations may also be derived, but it is far more difficult to develop a complete set of UML diagrams or models.

Completeness improves in direct proportion to the amount of analysis performed by the person doing reverse engineering. Interactivity refers to the degree to which the human is “integrated” with automated tools to create an effective reverse engineering process. In most cases, as the abstraction level increases, interactivity must increase or completeness will suffer.

If the directionality of the reverse engineering process is one-way, all information extracted from the source code is provided to the software engineer who can then use it during any maintenance activity. If directionality is two-way, the information is fed to a reengineering tool that attempts to restructure or regenerate the old program.

Before reverse engineering activities can commence, unstructured (“dirty”) source code is restructured so that it contains only the structured programming constructs. This makes the source code easier to read and provides the basis for all the subsequent reverse engineering activities.

The core of reverse engineering is an activity called extract abstractions. You must evaluate the old program and from the (often undocumented) source code, develop a meaningful specification of the processing that is performed, the user interface that is applied, and the program data structures or database that is used.

Reverse Engineering to Understand Data

Reverse engineering of data occurs at different levels of abstraction and is often the first reengineering task. At the program level, internal program data structures must often be reverse engineered as part of an overall reengineering effort. At the system level, global data structures (e.g., files, databases) are often reengineered to accommodate new database management paradigms (e.g., the move from flat file to relational or object-oriented database systems). Reverse engineering of the current global data structures sets the stage for the introduction of a new systemwide database.

Internal data structures. Reverse engineering techniques for internal program data focus on the definition of classes of objects. This is accomplished by examining the program code with the intent of grouping related program variables. In many cases, the data organization within the code identifies abstract data types. For example, record structures, files, lists, and other data structures often provide an initial indicator of classes.

Database structure. Regardless of its logical organization and physical structure, a database allows the definition of data objects and supports some method for establishing relationships among the objects. Therefore, reengineering one database schema into another requires an understanding of existing objects and their relationships.

Reverse Engineering to Understand Processing

Reverse engineering to understand processing begins with an attempt to understand and then extract procedural abstractions represented by the source code. To understand procedural abstractions, the code is analyzed at varying levels of abstraction: system, program, component, pattern, and statement.

The overall functionality of the entire application system must be understood before more detailed reverse engineering work occurs. This establishes a context for further analysis and provides insight into interoperability issues among applications within the system. Each of the programs that make up the application system represents a functional abstraction at a high level of detail. A block diagram, representing the interaction between these functional abstractions, is created. Each component performs some subfunction and represents a defined procedural abstraction. A processing narrative for each component is developed. In some situations, system, program, and component specifications already exist. When this is the case, the specifications are reviewed for conformance to existing code. Things become more complex when the code inside a component is considered. You should look for sections of code that represent generic procedural patterns. In almost every component, a section of code prepares data for processing, a different section of code does the processing, and another section of code prepares the results of processing for export from the component. Within each of these sections, you can encounter smaller patterns; for example, data validation and bounds checking often occur within the section of code that prepares data for processing.

For large systems, reverse engineering is generally accomplished using a semiautomated approach. Automated tools can be used to help you understand the semantics of existing code. The output of this process is then passed to restructuring and forward engineering tools to complete the reengineering process.

Reverse Engineering User Interfaces

Sophisticated GUIs have become de rigueur for computer-based products and systems of every type. Therefore, the redevelopment of user interfaces has become one of the most common types of reengineering activity. But before a user interface can be rebuilt, reverse engineering should occur. To fully understand an existing user interface, the structure and behavior of the interface must be specified. Merlo and his colleagues suggest three basic questions that must be answered as reverse engineering of the UI commences:

- What are the basic actions (e.g., keystrokes and mouse clicks) that the interface must process?
- What is a compact description of the behavioral response of the system to these actions?
- What is meant by a “replacement,” or more precisely, what concept of equivalence of interfaces is relevant here?

Behavioral modeling notation can provide a means for developing answers to the first two questions. Much of the information necessary to create a behavioral model can be obtained by observing the external manifestation of the existing interface. But additional information necessary to create the behavioral model must be extracted from the code.

It is important to note that a replacement GUI may not mirror the old interface exactly (in fact, it may be radically different). It is often worthwhile to develop a new interaction metaphor. For example, an old UI requests that a user provide a scale factor (ranging from 1 to 10) to shrink or magnify a graphical image. A reengineered GUI might use a slide-bar and mouse to accomplish the same function.

Software Test Document (STD)

The system test document specifies the tests for the entire software system, defines the test schedule, and records the test results. This document does not cover unit testing (the testing of individual sub-systems or components of the system).

The system may consist of multiple items that are to be tested separately. The system may be tested in one or more increments of functionality; the system test document should cover each version of the system separately. When different versions of the system are tested, make sure to clearly identify the version of the software and the relevant test and test result. Also, extend the unique identifier scheme to include the version of the software system under test (SUT) – for example, use TC-*vvvv*-*nnnn* to identify test case *nnnn* for software system version *vvvv*.

Template:

1 INTRODUCTION

1.1 System Overview

Briefly detail the software system and items to be tested. Identify the version(s) of the software to be tested.

1.2 Test Approach

Describe the overall approach to testing. For each major group of features or feature combinations, specify the approach that will ensure that these feature groups are adequately tested. Specify the major activities, techniques, and tools that are used to test the designated groups of features. The approach should be described in sufficient detail to permit identification of the major testing tasks and estimation of the time required to do each one. Identify significant constraints on testing, such as deadlines.

2 TEST PLAN

Describe the scope, approach, resources, and schedule of the testing activities. Identify the items being tested, the features to be tested, the testing tasks to be performed, the personnel responsible for each task in the case of a group project.

2.1 Features to be tested

Identify all software features and combinations of software features to be tested. Identify the test case(s) associated with each feature and each combination of features. Identify the version of the software to be tested.

When multiple versions of the software are tested in a planned, incremental manner, then use section numbers 2.1.*n* to identify the features to be tested

for each version.

2.2 Features not to be tested

Identify all features and significant combinations of features that will not be tested and the reasons for not doing so.

2.3 Testing Tools and Environment

Specify test staffing needs. For an individual project, specify the time to be spent on testing. For a group project, specify the number of testers and the time needed.

Specify the requirements of the test environment: space, equipment, hardware, software, special test tools. Identify the source for all needs that are not currently available.

3 TEST CASES

A test case specification refines the test approach and identifies the features to be covered by the case. It also identifies the procedures required to

accomplish the testing and specifies the feature pass/fail criteria. It documents the actual values used for input along with the anticipated outputs.

If an automated test tool is to be used:

- 1 document each test case here as a specification for the test tool;
.
- 2 document the procedure that must be followed to use the test tool.
.

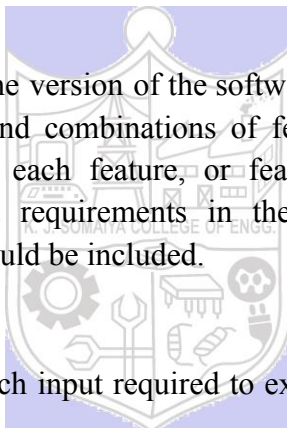
3.n Case-n (use a unique ID of the form TC-nnnn for this heading)

3.n.1 Purpose

Identify the version of the software and the test items, and describe the features and combinations of features that are the object of this test case. For each feature, or feature combination, a reference to its associated requirements in the software requirement specification (SRS) should be included.

3.n.2 Inputs

Specify each input required to execute the test case. Some of the inputs



will be specified by value (with tolerances where appropriate), while others, such as files or URLs, will be specified by name. Specify all required relationships between inputs (e.g., ordering of the inputs).

3.n.3 Expected Outputs & Pass/Fail criteria

Specify all of the expected outputs and features (e.g., response time) required of the test items. Provide the exact value (with tolerances where appropriate) for each required output or feature. Specify the criteria to be used to determine whether each test item has passed or failed testing. If an automated test tool is used, identify how the results of that tool are to be analysed.

3.n.4 Test Procedure

Detail the test procedure(s) needed to execute this test case. Describe any special constraints, such as: special set up, operator intervention, output determination procedures, and special wrap up.

4. Test Logs

A test log is used by the test team to record what occurred during test execution. A.n

Log for test-n (use a unique ID of the form TL-nnnn for this heading)

5. Test Results

For each execution, record the date/time and observed results (e.g., error messages generated). Also record the location of any output (e.g., window on the screen). Record the successful or unsuccessful execution of the test.

Incident Report (add a unique ID of the form TIR-nnnn to this heading) If the test failed, or passed with some unusual event, fill in this incident report with the details. Summarize the incident, identifying the test items involved, and the anomaly in the results. Indicate what impact this incident will have on the project.